UNIT 3 — CNN, Object Detection, Segmentation

What is a CNN?

A Convolutional Neural Network (CNN) is a neural network architecture designed to learn patterns from data with spatial structure. Images are the most common example.

A pixel is not independent of its neighbors. A small region may form an edge, corner, texture or object part. CNNs exploit this local structure using learnable filters.

```
Image
  ↓
Convolution
  ↓
Feature maps
  ↓
Activation (e.g., ReLU)
  ↓
Pooling / Downsampling
  ↓
More convolution blocks
  ↓
Flatten / Global pooling
  ↓
Classifier
  ↓
Class probabilities
```

Why Not Just Use a Fully Connected Network?

- A large image contains a very large number of pixels and therefore creates an enormous number of parameters in a fully connected layer.
- Fully connected layers do not naturally exploit the local spatial relationship between neighboring pixels.
- CNNs use local receptive fields and shared filter weights, which makes them much more parameter-efficient.
- The same filter can detect a useful pattern at different positions in the image.

Convolution Operation

A convolution layer applies a small matrix called a kernel or filter across an input. At each location, corresponding values are multiplied and summed.

```
Example input:
1 2 3
4 5 6
7 8 9

Example 2×2 filter:
1  0
0 -1

At the top-left:
(1×1) + (2×0) + (4×0) + (5×-1)
= -4
```

The filter slides over the image and produces a new matrix called a feature map.

What Does a CNN Filter Learn?

In classical computer vision, we often design filters manually. Examples include Sobel edge filters, Laplacian filters and Gaussian filters. In a CNN, filter values are learned from training data through backpropagation and optimization.

```
Early layers  → edges, simple textures
Middle layers → corners, motifs, object parts
Deep layers   → semantic structures and object-level patterns
```

Important distinction: The CNN is not explicitly told 'this filter is an edge detector'. Training discovers filter values that help minimize the task loss.

Kernel, Filter and Channel

For a color image, the input may have three channels: red, green and blue. A convolution filter spans all input channels.

```
Input: 32 × 32 × 3
Number of filters: 16
Kernel: 3 × 3

Output: 32 × 32 × 16   (with stride 1 and padding 1)
```

Each of the 16 filters produces one output feature map. Therefore, 16 filters create 16 output channels.

Padding

Padding adds values around the border of the input, commonly zeros. Without padding, repeated convolutions continuously shrink the spatial dimensions.

```
Input 32×32
3×3 convolution, stride 1, padding 0
Output = 30×30

Input 32×32
3×3 convolution, stride 1, padding 1
Output = 32×32
```

Why padding?: It preserves spatial dimensions when desired and gives border pixels more opportunity to influence learned representations.

Stride

Stride specifies how many pixels the kernel moves after each operation.

```
Stride 1 → moves one pixel at a time
Stride 2 → moves two pixels at a time
```

A larger stride usually reduces spatial resolution and computation.

Convolution Output Size

For one spatial dimension:

```
Output size = floor((N - F + 2P) / S) + 1
```

N = input size, F = filter size, P = padding, S = stride.

Example: N=32, F=3, P=1, S=1 gives 32.

```
floor((32 - 3 + 2×1)/1) + 1 = 32
```

Feature Maps

A feature map is the response produced by a filter as it scans the input. Strong activations indicate locations where the learned pattern is strongly present.

```
Input image
    ↓
Filter 1 → Feature map 1
Filter 2 → Feature map 2
Filter 3 → Feature map 3
...
Filter K → Feature map K
```

ReLU After Convolution

A common CNN block is convolution followed by a nonlinear activation such as ReLU.

```
ReLU(x) = max(0, x)
```

Without nonlinear activations, stacking linear operations would remain effectively linear and would severely limit what the network can learn.

Practical — Convolution with OpenCV

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

# Vertical edge detector
kernel = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
], dtype=np.float32)

edge = cv2.filter2D(image, -1, kernel)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(edge, cmap="gray")
plt.title("Filter Response")
plt.axis("off")
plt.show()
```

Practical — PyTorch Conv2d

```
import torch
import torch.nn as nn

x = torch.randn(1, 3, 32, 32)

conv = nn.Conv2d(
    in_channels=3,
    out_channels=16,
    kernel_size=3,
    stride=1,
    padding=1
)

y = conv(x)

print("Input :", x.shape)
print("Output:", y.shape)

# Output:
# Input : torch.Size([1, 3, 32, 32])
# Output: torch.Size([1, 16, 32, 32])
```

POOLING OPERATIONS

Why Pooling?

Pooling reduces the spatial resolution of feature maps. It reduces computation and can make representations less sensitive to small translations.

```
32 × 32 × 64
      ↓
2×2 pooling, stride 2
      ↓
16 × 16 × 64
```

Key point: Pooling changes height and width, but normally does not change the number of channels.

Max Pooling

```
Input:
1  3  2  4
5  6  1  2
7  2  8  3
4  1  2  9

2×2 max pooling, stride 2:

6  4
7  9
```

Max pooling keeps the strongest activation in each local region.

Average Pooling

Average pooling computes the mean of values inside each pooling window.

```
For:
1 3
5 6

Average = (1+3+5+6)/4 = 3.75
```

Max vs Average Pooling

Property	Max Pooling	Average Pooling
Operation	Maximum	Mean
Typical effect	Keeps strongest response	Smooths local response
Common use	Feature downsampling	Downsampling / global average pooling
Intuition	Is the feature strongly present?	What is the average activation?

Practical — Pooling in PyTorch

```
import torch
import torch.nn as nn

x = torch.randn(1, 3, 32, 32)

max_pool = nn.MaxPool2d(kernel_size=2, stride=2)
avg_pool = nn.AvgPool2d(kernel_size=2, stride=2)

print(max_pool(x).shape)
print(avg_pool(x).shape)

# Both:
# torch.Size([1, 3, 16, 16])
```

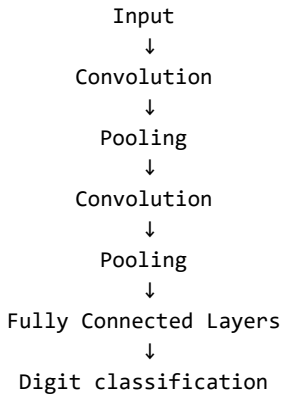
CLASSICAL CNN ARCHITECTURES

LeNet-5

LeNet-5 (1998) was designed by Yann LeCun for handwritten digit recognition, especially MNIST-like images.

Architecture:

Input 32×32×1
→ **C1: Conv 5×5, 6 filters**
→ **S2: Average Pooling 2×2**
→ **C3: Conv 5×5, 16 filters**
→ **S4: Average Pooling 2×2**
→ **C5: Conv 5×5, 120 filters**
→ **F6: Fully Connected 84**
→ **Output: 10 classes**



Its importance is conceptual: it demonstrated that convolutional feature extraction followed by learned classification can work extremely well for visual recognition tasks.

LeNet-Style Practical Model

```
import torch
import torch.nn as nn

class LeNet(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(1, 6, 5),
            nn.ReLU(),
            nn.AvgPool2d(2),

            nn.Conv2d(6, 16, 5),
            nn.ReLU(),
            nn.AvgPool2d(2)
        )

        self.classifier = nn.Sequential(
```

```
            nn.Flatten(),
            nn.Linear(16 * 4 * 4, 120),
            nn.ReLU(),
            nn.Linear(120, 84),
            nn.ReLU(),
            nn.Linear(84, 10)
        )

    def forward(self, x):
        return self.classifier(self.features(x))

model = LeNet()
print(model)
```

AlexNet

AlexNet was a landmark CNN that demonstrated the power of deep learning at large scale for image classification.

- Used ReLU activations extensively.
- Made effective use of GPU computation.
- Used dropout for regularization.
- Used data augmentation.
- Contained substantially more learned capacity than earlier CNNs.

Historical significance: AlexNet helped establish deep CNNs as a dominant approach to large-scale visual recognition.

Architecture:

Input 227×227×3
→ **Conv1: 11×11, 96**
→ **Max Pool**
→ **Conv2: 5×5, 256**
→ **Max Pool**
→ **Conv3: 3×3, 384**
→ **Conv4: 3×3, 384**
→ **Conv5: 3×3, 256**
→ **Max Pool**
→ **FC 4096**
→ **FC 4096**
→ **Output 1000**

Practical — Loading AlexNet

```
import torch.nn as nn
from torchvision.models import alexnet, AlexNet_Weights

model = alexnet(weights=AlexNet_Weights.DEFAULT)

# Example: adapt to 10 classes
model.classifier[6] = nn.Linear(
    model.classifier[6].in_features,
    10
)

print(model)
```

ADVANCED CNN ARCHITECTURES

VGG-16

VGG-16 is a deep convolutional neural network (CNN) architecture introduced by Karen Simonyan and Andrew Zisserman from the Visual Geometry Group (VGG) at the University of Oxford in 2014.

Input Layer: 224 × 224 × 3 (RGB image)

1. **Block 1: Two Conv2D layers (64 filters, 3 × 3), followed by MaxPool (2 × 2)**
2. **Block 2: Two Conv2D layers (128 filters, 3 × 3), followed by MaxPool (2 × 2)**
3. **Block 3: Three Conv2D layers (256 filters, 3 × 3), followed by MaxPool (2 × 2)**
4. **Block 4: Three Conv2D layers (512 filters, 3 × 3), followed by MaxPool (2 × 2)**
5. **Block 5: Three Conv2D layers (512 filters, 3 × 3), followed by MaxPool (2 × 2)**
6. **Fully Connected Layer 1 (Dense): 4,096 channels**
7. **Fully Connected Layer 2 (Dense): 4,096 channels**
8. **Output Layer (Dense/Softmax): 1,000 channels (for ImageNet classification)**

Main idea: Use repeated small convolutions to build a deep hierarchical representation.

Limitation: VGG is computationally and parameter intensive, especially compared with later efficient designs.

Challenges in Deep Neural Networks

Deep Neural Networks are powerful models, but training them becomes difficult as network depth increases. Two major issues are:

1. Vanishing/Exploding Gradient Problem: As the number of layers increases, gradients can become extremely small (vanishing) or very large (exploding) during backpropagation, making training unstable.

2. Degradation Problem: Increasing network depth does not always improve performance and can even degrade it.

ResNet

- A **residual neural network** (also referred to as a **residual network** or **ResNet**) is a deep learning architecture in which the layers learn residual functions with reference to the layer inputs.
- It was developed in 2015 for image recognition, and won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) of that year.
- ResNet introduced residual or skip/residual connections to make very deep networks easier to optimize minimizing the degradation problem.



In a typical ResNet basic block, there are two convolutional layers, Batch Normalization, activation functions, and the shortcut connection.

Why it helps: Skip connections which makes it possible to train very deep neural networks by allowing information and gradients to bypass some layers.

Practical — ResNet

```
from torchvision.models import resnet18, ResNet18_Weights
import torch.nn as nn

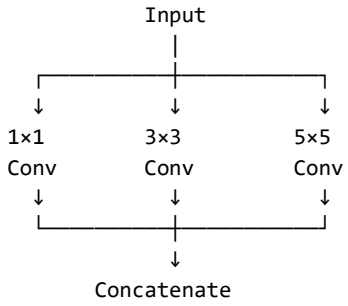
model = resnet18(weights=ResNet18_Weights.DEFAULT)

model.fc = nn.Linear(
    model.fc.in_features,
    10
)

print(model)
```

Inception Networks (GoogLeNet)

The Inception idea is to process information at multiple spatial scales inside a block.



This allows the network to learn patterns at different scales rather than committing to a single kernel size.

Useful when want to learn diverse image features efficiently.

MobileNet

A conventional CNN that uses standard convolution layers perform two jobs simultaneously:

- Extract spatial features, such as edges, textures, and shapes.
- Combine information from different input channels.

Doing both jobs together can be computationally expensive, particularly on mobile phones and embedded devices.

MobileNet addresses this using a technique called depthwise separable convolution.

```
Standard convolution:
Input → full convolution → Output


MobileNet style Architecture (Depthwise separable):
Input
 ↓
Depthwise convolution
 ↓
Pointwise 1×1 convolution
 ↓
Output
```

Depthwise convolution processes spatial information channel-by-channel, i.e each input channel gets its own spatial filter.

For example:
Input channels = 32
Filters per input channel = 1
Filter size = 3 * 3

So each of the 32 channels is processed independently. It does not combine information across channels. It only extracts spatial features within each channel.

Now we need to combine the information from 32 channels and generate 64 output channels if we have 64 filters in convolution layer for MobileNet uses a 1×1 convolution, called pointwise convolution which mixes information across channels.

Advantages of using MobileNet:

Suppose input feature map diemsion : 112 * 112 * 32 and we want to produce 64 output channels using a 3*3 convolution.

In standard convolution, each filter has dimensions: 3*3*32 and 64 filters are there

So total weights = $3 \times 3 \times 32 \times 64 = 18{,}432$

For depthwise 3 * 3 is size of each filter and we have 32 channels

The number of weights is: $3 \times 3 \times 32 = 288$

And now 1 by 1 pointwise convolution will be used across all 32 channels to combine information

And we want 64 channels output → $1 \times 1 \times 32 \times 64 = 2{,}048$ weights

Practical — MobileNet

```
from torchvision.models import mobilenet_v2, MobileNet_V2_Weights
import torch.nn as nn

model = mobilenet_v2(weights=MobileNet_V2_Weights.DEFAULT)

model.classifier[1] = nn.Linear(
    model.classifier[1].in_features,
    10
)

print(model)
```

Architecture Comparison

Architecture	Core idea	Problem / goal
LeNet-5	Early convolution + pooling pipeline	Learn visual features for recognition
AlexNet	Deep CNN + ReLU + GPU + dropout	Large-scale image classification
VGG-16	Stacks of 3×3 convolutions	Simple, deep architecture
ResNet	Residual/skip connections	Train very deep networks
Inception	Parallel multi-scale branches	Capture features at multiple scales
MobileNet	Depthwise separable convolution	Reduce computation for efficient deployment

OBJECT DETECTION

Classification vs Detection vs Segmentation

Task	Output	Example
Classification	One or more class labels	Image → dog
Object detection	Class + bounding box for each object	dog + coordinates
Segmentation	Identifies the pixels belonging to an object or category, providing more detailed boundaries than a bounding box.	Each pixel assigned to a region/class

Simple Example

Imagine a photo containing dog and grass.

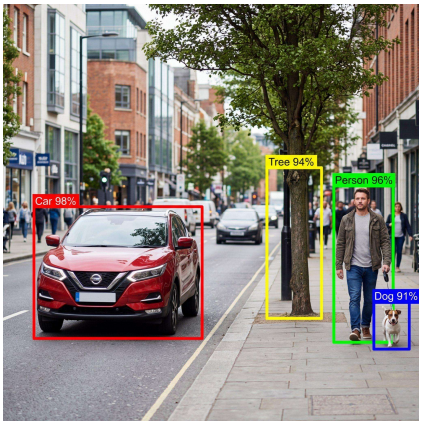
For image classification it identifies that image contains a dog.

For Object Detection: Draws a boundary box around the dog.

For Image Segmentation it identifies the exact pixels belonging to dog, separating it from the grass and background.

Object detection is a computer vision task in which a model identifies what objects are present in an image and where each object is located.

For example, suppose you give a model an image containing a person, a car, and a dog.



The model might produce these predictions:

Object	Class	Bounding box	Confidence
Person	Person	Coordinates around the person	0.97
Car	Car	Coordinates around the car	0.94
Dog	Dog	Coordinates around the dog	0.91

Bounding Boxes

A bounding box is a rectangle that surrounds a detected object.
It identifies the approximate location of an object.

Representation A: Corner coordinates

$$(x_{\min}, y_{\min}, x_{\max}, y_{\max})$$

- $x_{\min}$: left edge
- $y_{\min}$: top edge
- $x_{\max}$: right edge
- $y_{\max}$: bottom edge

For example:

$$(100, 50, 300, 250)$$

This means that the box starts at pixel $(100, 50)$ and ends at pixel $(300, 250)$.

$$\texttt{Width} = 300 - 100 = 200$$

$$\texttt{Height} = 250 - 50 = 200$$

Representation B: Center coordinates

Coordinates are often normalized between 0 and 1 in detection models or stored in pixel coordinates in datasets.

$$(x_c, y_c, w, h)$$

Here:

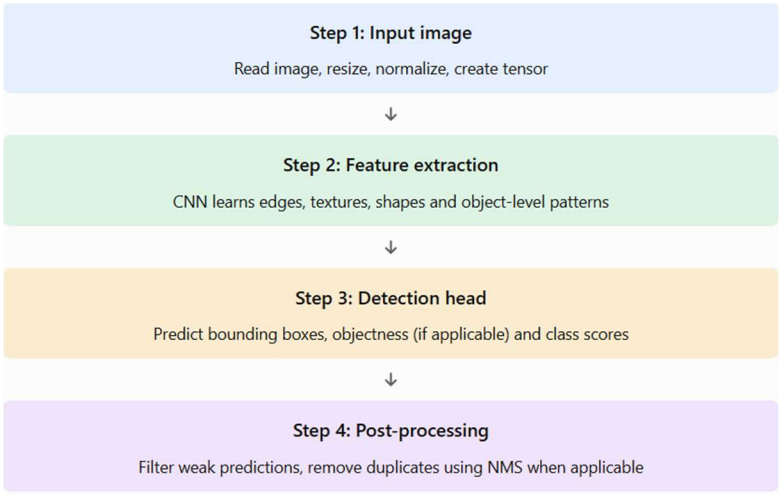
- x_c: horizontal center
- y_c: vertical center
- w: box width
- h: box height

The conversion from corner coordinates is:

$$x_c = \frac{x_{\min} + x_{\max}}{2}$$

$$y_c = \frac{y_{\min} + y_{\max}}{2}$$

Complete Flow of working of object detection model



The detection head predicts information about possible objects.

For each candidate detection, it may estimate:

- Box coordinates.
- Objectness or object-presence score, depending on the architecture.
- Class scores, such as person, dog or car.

Non-Maximum Suppression (NMS)

NMS is a common method for eliminating redundant detections.

Its central idea is:

Keep the highest-scoring box and suppress other sufficiently overlapping boxes that likely describe the same object.

Main types of object detection algorithms

Two-stage detectors

Examples: R-CNN, Fast R-CNN, Faster R-CNN.

First generate candidate object regions, then classify and refine them.

Strength: Often strong accuracy, though speed varies by model and hardware.

One-stage detectors

Examples: YOLO family, SSD, RetinaNet.

Detects objects by predicting bounding boxes and class probabilities in a single forward pass through a neural network.

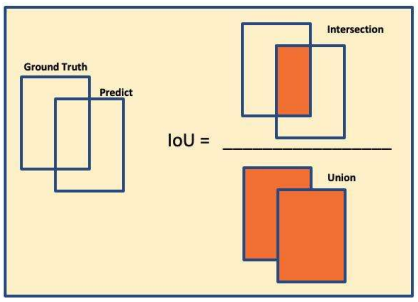
Strength: Often suitable for real-time applications.

IoU — Intersection over Union

IoU measures how much a predicted bounding box overlaps with the actual/ground-thruth bounding box of an object.

In simple words, it tells us how accurately the model has located an object.

```
IoU = Area of Intersection / Area of Union
```



Actual box (Ground Truth): Where the dog really is.

Predicted box: Where the AI thinks the dog is.

Overlap: The area shared by both boxes.

IoU ranges from 0 to 1. A value of 1 means perfect overlap; 0 means no overlap.

Why IoU matters: It is used in detection evaluation and in deciding whether a prediction sufficiently overlaps a ground-truth object.

Example

Suppose:

- Actual box area = 100 pixels²
- Predicted box area = 80 pixels²
- Overlapping area = 60 pixels²

Union area:

$$100 + 80 - 60 = 120$$

Therefore,

$$\text{IoU} = \frac{60}{120} = 0.5$$

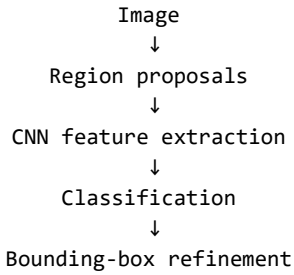
So, **IoU = 0.5, or 50% overlap.**

R-CNN

R-CNN means Regions with CNN features.

Instead of analysing the entire image as one classification problem, generate candidate regions and determine which objects are present in each region.

R-CNN first generates possible regions where objects may exist. It then analyses those regions individually and predicts their classes and bounding boxes.



Later descendants such as Fast R-CNN and Faster R-CNN improved speed and integrated more of the pipeline.

Step 1: Input image

The image may contain multiple objects, background areas, and objects of different sizes.

The original R-CNN does not immediately produce detections from the entire image. It first needs to determine which parts of the image should be examined.

Step 2: Region proposal generation using Selective Search

How Selective Search works

1. Initial segmentation: Divide the image into small regions based on visual similarities.
2. Region comparison: Compare neighboring regions using properties such as color, texture, size, and shape compatibility.
3. Region merging: Merge similar neighboring regions into larger regions.
4. Candidate generation: Record regions at different scales as possible objects.

Using Selective Search, R-CNN generates approximately 2,000 candidates regions from a single input image maybe many of this 2000 regions overlap each other.

Some proposals (candidates regions) may contain an entire car, while others may contain only a wheel, a portion of the background, or multiple objects.

The purpose is to generate candidate regions likely to include objects.

Why are region proposals needed?

Imagine a 640 × 640 image. It contains hundreds of thousands of possible rectangular boxes. Evaluating every possible rectangle using a CNN would be computationally expensive.

Selective Search reduces this search space to a manageable number of candidates.

Step 3: CNN feature extraction

After generating proposals, R-CNN extracts visual features from each region using a pretrained CNN, such as AlexNet, and fine-tuned for the detection task.

Process

1. Crop each proposed region from the original image.
2. Warp or resize it to the fixed input dimensions required by the CNN.
3. Pass it through the convolutional and pooling layers to detect useful features.
4. Extract a fixed-length feature vector.

The vector is then passed to the classification and bounding-box regression components.

Step 4: Object classification using Binary SVM classifier

Now the extracted features are given to class-specific binary Support Vector Machines (SVMs) classifiers

Suppose the dataset contains three object classes:

- Car / Person / Bicycle

R-CNN trains one binary SVM for each object class, plus a background treatment for regions that do not correspond to a target object.

Each SVM uses the CNN feature vector received after passing image to pretrained CNN model to calculate a class score.

For example:

Candidate region	Car score	Person score	Bicycle score
Region A	2.8	0.3	0.1
Region B	0.2	3.1	0.4
Region C	0.4	0.5	2.7

Step 5: Bounding-box regression

Now the regression model adjusts the box coordinates by applying some transformations which adjust the center, width, and height of the original proposal.

This is done soo that the box fits the detected object more accurately.

Let the proposal be:

$$P = (x, y, w, h)$$

where x, y are the center coordinates, and w, h are its width and height.

The regression model predicts adjustments:

$$(t_x, t_y, t_w, t_h)$$

Step 6: Non-Maximum Suppression

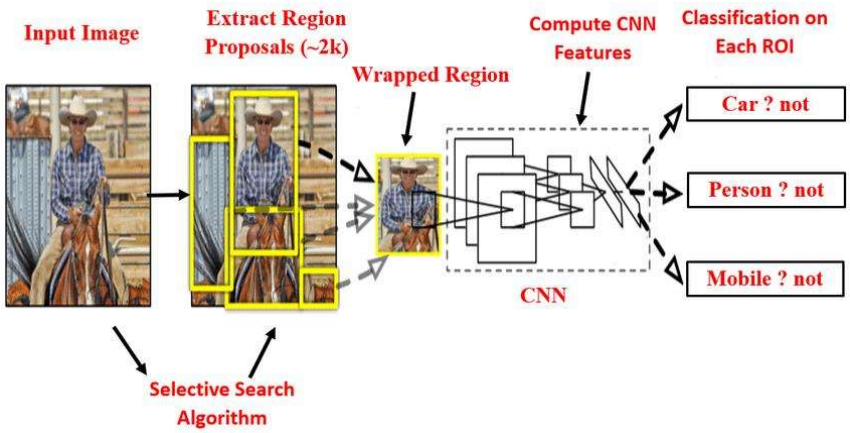
Different region proposals may identify the same object.

For example, five proposals might all detect the same car, without post-processing, the model could output five boxes for one car.

NMS reduces these duplicate predictions:

1. Sort detections by their class scores.
2. Select the highest-scoring box.
3. Calculate its IoU with other boxes of the same class.
4. Suppress boxes whose overlap exceeds the chosen threshold.
5. Repeat with the remaining boxes.

The final output contains a reduced set of detections.



Advantages and disadvantages of R-CNN

Advantages	Disadvantages
CNNs learn useful visual features automatically.	The original method runs a CNN separately on many proposals.
Region-based processing can localize objects effectively.	Inference is computationally expensive.
Bounding-box regression improves localization.	Selective Search adds processing time.
It established a foundation for later region-based detectors.	Training involves multiple separate stages.
Useful for understanding two-stage detection concepts.	High memory and storage requirements for extracted features.

YOLO — You Only Look Once

Instead of generating thousands of candidate regions and processing them separately, YOLO predicts object locations, object confidence, and class probabilities from the image in a single forward pass through the network. This is why YOLO is called a one-stage object detector.

Step 1: Input Image

Example: One image containing a dog, a cat, and a car.

Step 2: Image Preprocessing

The image is resized to the dimensions required by the YOLO model and its pixel values are prepared for input.

Step 3: CNN / Backbone

The network processes the image and extracts useful features such as edges, shapes, textures, and object parts.

Step 4: Detection Head

The model predicts bounding box coordinates and class scores for possible objects. Depending on the YOLO version, predictions may be made at multiple feature-map locations and scales.

Step 5: Confidence Filtering

Predictions with scores below the chosen threshold are discarded.

Step 6: Non-Maximum Suppression (NMS)

For overlapping predictions of the same object, redundant boxes are removed when NMS is used.

Step 7: Final Output

The image with detected objects, class labels, confidence scores, and bounding boxes.

Anchor Boxes

An anchor box is a predefined rectangular box placed at different locations in an image to help an AI model detect and locate objects.

Imagine an image containing a dog. The AI needs to answer two questions:

1. What is the object? — Dog.
2. Where is the object? — The position and size of the dog.

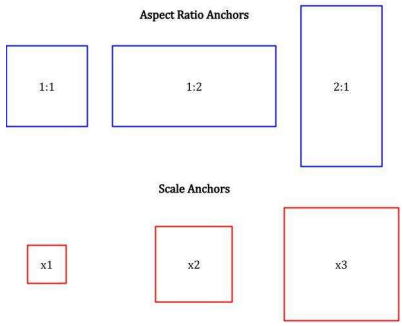
Anchor boxes help the model predict the object's location and size.

Objects have different shapes and sizes.

A model might use anchor boxes with different aspect ratios:

- 1:1 — square
- 2:1— wide
- 1:2 — tall

It can also use different sizes to detect small, medium, and large objects.



Anchor box	Bounding box
Predefined reference box	Box representing the detected object
Exists before the model's box adjustments	Produced after the model predicts adjustments
May not fit the object well	Ideally fits the object closely

Analogy: An anchor box is like a ready-made frame, and the model adjusts that frame to fit the object.

IMAGE SEGMENTATION

What is Image Segmentation?

Image segmentation means dividing an image into meaningful regions or assigning a class to every pixel in the image.

Imagine an image containing a dog. An AI model should not only identify that a dog exists; it should also identify exactly which pixels belong to the dog.

Object detection asks, “Where is the object?” Image segmentation asks, “Which exact pixels belong to each region or object?”

```
Image
 ↓
Segmentation model
 ↓
Pixel-wise mask

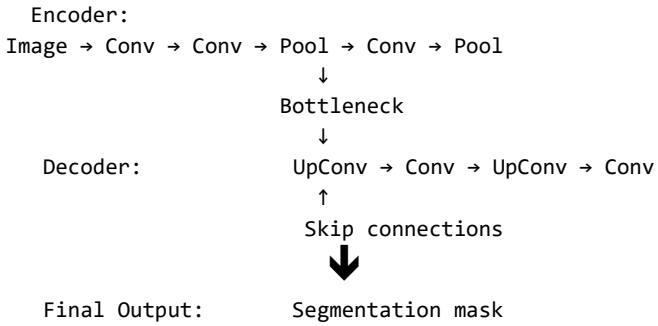
Example:
0 0 1 1 1
0 0 1 1 1
0 2 2 2 1
0 2 2 2 0
```

Here, different integer values can represent background and different object classes.

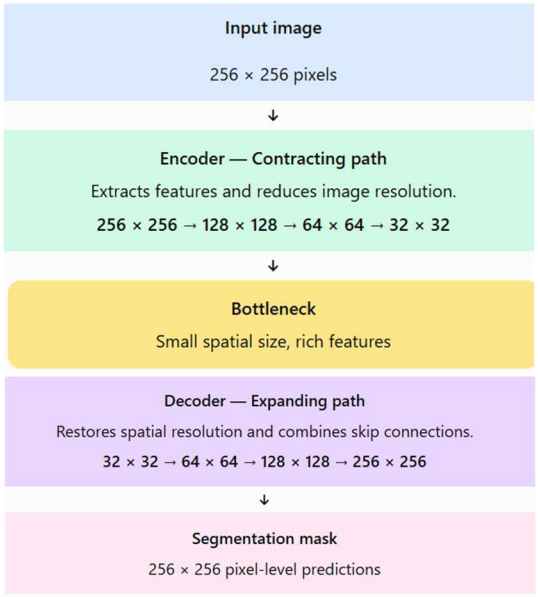
U-Net

U-Net is a deep learning neural network architecture designed mainly for image segmentation. It learns to identify which pixels belong to the target object or class.

For example, suppose a doctor gives the model an MRI scan. U-Net can learn to mark the pixels corresponding to a tumor, helping outline its location and shape.



Skip connections transfer feature maps directly from the encoder to matching decoder stages. The decoder combines those details with its own features to make more accurate segmentation masks by avoiding the problem of loss if information.



For binary segmentation, the model predicts a probability for every pixel.

For multiclass segmentation, the model can instead predict among classes such as background, healthy tissue, and tumor.

EXPLAINABLE COMPUTER VISION

Suppose you give a deep learning model an image of a dog. The model predicts:

Prediction: Dog — 98% confidence

But how do we know why it predicted a dog. Did it focus on the dog's face, its ears, its body, or the grass behind it.

Explainable computer vision methods attempt to identify image regions or input features that contributed to a prediction.

This is especially useful for debugging models, finding spurious correlations and building trust in high-stakes applications.

Guided Backpropagation

Backpropagation computes gradients of a target output with respect to earlier values.

Guided backpropagation modifies how gradients are passed through ReLU layers so that positive contributions are emphasized.

Unlike ordinary training backpropagation, its primary goal here is visualization, not updating the model's weights.

```
Prediction
   ↓
Target score
   ↓
Backpropagation
   ↓
Input gradients
   ↓
Visualization
```

The ReLU function is:

$$f(x) = \max(0, x)$$

In the forward pass:

- If $x > 0$, the output is x.
- If $x \leq 0$, the output is 0.

During standard backpropagation, the gradient through ReLU is zero when the forward input is negative and is normally one when it is positive (away from the zero boundary).

Guided Backpropagation adds another condition: the backward gradient must also be positive.

Its modified gradient rule can be written as:

$$g_{\text{guided}} = \begin{cases} g_{\text{out}}, & x > 0 \text{ and } g_{\text{out}} > 0 \\ 0, & \text{otherwise} \end{cases}$$

Here:

- x is the input to the ReLU.
- g_{out} is the gradient arriving from the layer above.
- g_{guided} is the gradient passed backward.

A gradient passes only when both the forward activation and the incoming gradient are positive.

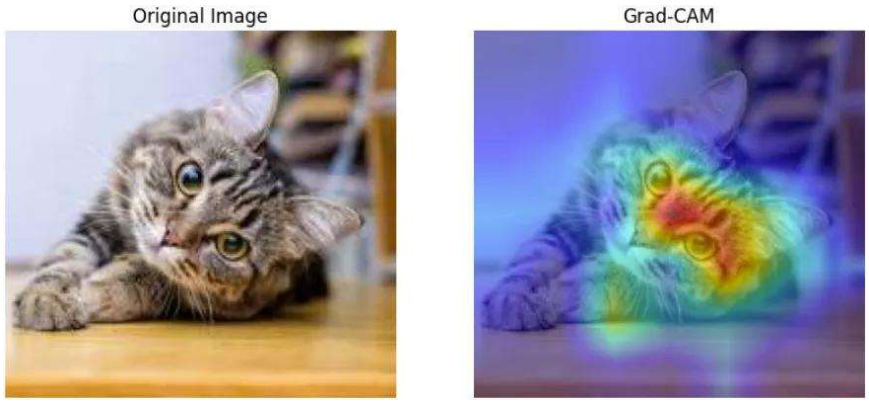
The resulting visualization can highlight pixels that strongly influence the chosen output.

CAM — Class Activation Mapping

Class Activation Mapping (CAM) is a technique for identifying the image regions that contribute to a CNN's classification of a particular class.

Unlike Guided Backpropagation, which often highlights fine edges and details, CAM creates a more spatially meaningful heatmap indicating where class-relevant features are located.

Imagine a model classifying an image as a bird. CAM can show whether the model focused on the bird's body or on the surrounding background.



- Red or yellow regions in many visualizations indicate stronger class-related activation.
- Blue or dark regions indicate weaker activation.

The conceptual flow is:

```
Feature maps from final convolutional layer then Apply Global Average Pooling to each feature map.
                ↓
Class-specific weighted combination
                ↓
Activation map
                ↓
Upsample to image size
                ↓
Heatmap
```

The limitation of CAM

The original CAM technique requires a specific architecture: the convolutional feature maps must connect to the classifier through global average pooling and a linear classification layer.

This architecture requirement can make CAM inconvenient for networks that do not have this structure.

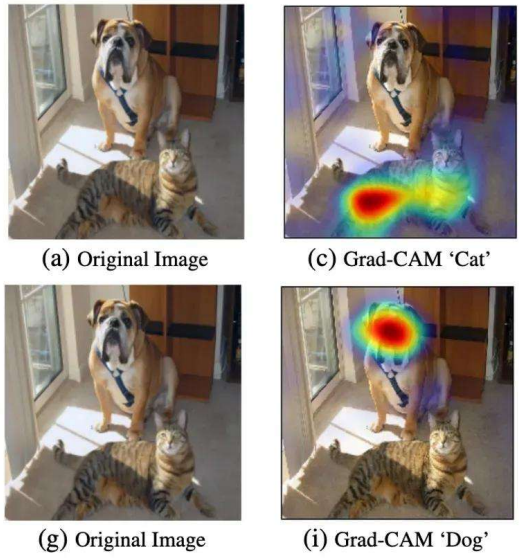
This limitation motivated Grad-CAM, which can be applied to a wider variety of CNN architectures.

Grad-CAM

Grad-CAM generalizes the idea by using gradients flowing into a convolutional feature layer to determine which feature maps are important for a target class.

It creates a heatmap that highlights regions supporting the selected class.

For example, if a model predicts "cat", Grad-CAM may highlight the cat's face and body. If we select "dog" instead, it may generate a different map for the same image.



(a) Original Image (c) Grad-CAM 'Cat'

(g) Original Image (i) Grad-CAM 'Dog'

```
Input image
    ↓
   CNN
    ↓
Target class score
    ↓
Gradients with respect to selected feature maps
    ↓
Weighted combination of feature maps
    ↓
ReLU
    ↓
Heatmap
    ↓
Overlay on image
```

Interpretation: Red/hot regions in a Grad-CAM visualization are not automatically proof of causality. They are evidence of which spatial features contributed strongly to the selected model output under the method's assumptions.

ADVERSARIAL ATTACKS

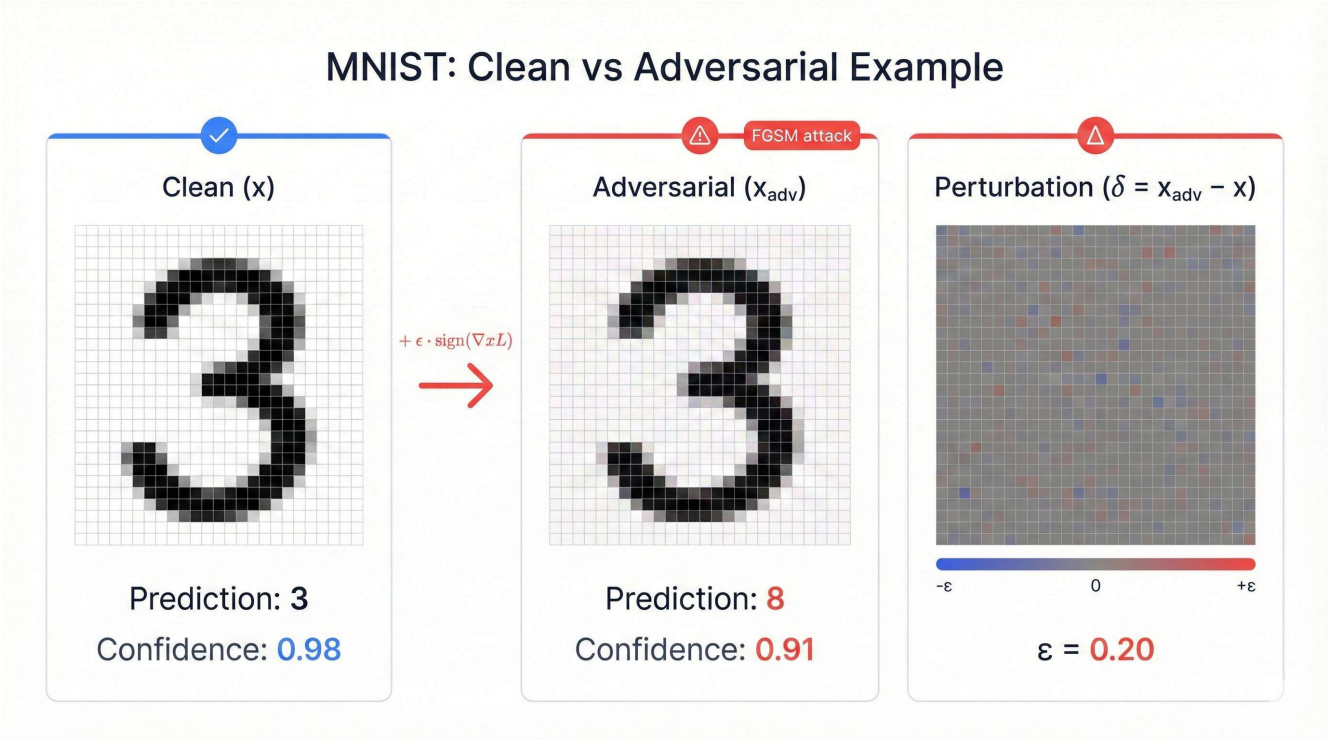
An adversarial attack is an attempt to fool a machine learning model by giving it an input that has been deliberately modified by a small perturbation so that a machine-learning model makes an incorrect prediction or behaves in an undesired way.

In computer vision, an attacker may change an image by a very small amount. The image may still look almost identical to a human, but the model's prediction can change significantly.

```
Original image
      ↓
CNN → correct prediction

Original image + small perturbation
      ↓
CNN → incorrect / targeted prediction
```

The perturbation may be visually subtle to a human while significantly changing the model's output.



AI models learn **patterns/features from data**. Sometimes they rely on features that humans don't notice.

An attacker tries to find a tiny change:

Original input + small perturbation → wrong prediction

Types of adversarial attacks

A. White-box attack

The attacker knows the model's architecture and has access to its parameters or gradients.

This knowledge can be used to calculate perturbations that target the model's prediction.

B. Black-box attack

A Black-Box Attack is an adversarial attack in which the attacker does not have access to the internal details of a machine learning model, such as its architecture, weights, parameters, or gradients.

Instead, the attacker tries to understand or fool the model by observing its inputs and outputs.

C. Targeted attack

The attacker tries to make the model predict a specific incorrect class.

For example:

- Actual object: Cat.
- Intended adversarial prediction: Dog.

D. Untargeted attack

An untargeted attack only tries to make the model predict any class other than the correct class.

For example:

- Actual object: Cat.
- Adversarial prediction: Fox.

The attacker does not necessarily care which wrong class is predicted.

- Targeted = Target fixed. Untargeted = Any wrong class.

FGSM — Fast Gradient Sign Method

FGSM is a classic gradient-based adversarial attack introduced by Goodfellow and colleagues.

FGSM (Fast Gradient Sign Method) is a gradient-based adversarial attack used to fool a machine learning or deep learning model by making a small change to its input.

The main idea is:

Change the input in the direction that increases the model's loss, so the model is more likely to make an incorrect prediction.

FGSM is a one-step attack.

The formula for an untargeted FGSM attack is: $x_{adv} = x + \varepsilon \cdot \text{sign}(\nabla x\, J(\theta, x, y))$

x is the original input,
ε controls perturbation magnitude,
J is the loss,
θ represents model parameters, and
y is the true label.

Why Adversarial Attacks Matter

- They reveal that high test accuracy does not necessarily mean robustness.
- They help researchers evaluate model reliability under perturbations.
- They motivate adversarial training and other robustness techniques.
- They demonstrate that neural networks can rely on patterns that are not obvious to humans.

QUICK SUMMARY

CNN Fundamentals

- Convolution → extracts learned local patterns.
- Padding → controls border treatment and spatial size.
- Stride → controls movement of the kernel and downsampling.
- Feature map → output activation map of a filter.
- ReLU → introduces nonlinearity.

Pooling

- Max pooling → strongest local activation.
- Average pooling → local mean.
- Pooling usually reduces height and width.

Architectures

- LeNet → early practical CNN.
- AlexNet → large-scale deep CNN breakthrough.
- VGG → repeated 3×3 convolutions.
- ResNet → residual/skip connections.
- Inception → parallel multi-scale processing.
- MobileNet → efficient depthwise separable convolution.

Detection

- Bounding box → object localization.
- IoU → intersection area divided by union area.
- R-CNN → region proposals + CNN features.
- YOLO → unified real-time-oriented detection.
- Anchor boxes → predefined reference boxes used by many detectors.

Segmentation

- Segmentation → pixel-level prediction.
- U-Net → encoder-decoder with skip connections.

Explainability

- Guided Backpropagation → gradient-based visualization.
- CAM → class-specific activation map under suitable architecture.
- Grad-CAM → uses gradients and convolutional feature maps to localize important regions.

Adversarial Attacks

- Adversarial example → deliberately perturbed input that changes model behavior.
- FGSM → gradient-sign based perturbation.